ferro Documentation

ferro developers

Contents

Introduction

ferro is a Rust library and command-line toolkit for post-processing molecular dynamics (MD) trajectories and computational chemistry data. It is designed for periodic systems (crystals, surfaces, glasses) and covers:

- **Structural analysis** — g(r), S(q), bond angle distributions, Qn speciation
- **Dynamical analysis** — MSD, VACF, van Hove correlation, rotational correlation
- **Spatial maps** — 3-D density/velocity/force grids, jump-distance maps, hard-sphere occupancy, cluster SDF
- **Charge analysis** — Bader decomposition (on-/near-/off-grid, Yu-Trinkle weight), averaged charge-density SDF
- **Input generation** — Gaussian, CP2K, and Quantum ESPRESSO input files with structure-based spin estimation and a precise CP2K basis/pseudopotential database
- **I/O** — readers and writers for LAMMPS dump, VASP POSCAR/OUTCAR/CHGCAR, XYZ, PDB, CIF, QE, CP2K, Gaussian cube
- **Python bindings** — `import ferro`: read/write, supercell/vacuum/merge, g(r) & MSD (details)

Design Goals

Goal	Implementation
Correct periodic-boundary handling	Minimum-image convention; fractional-coordinate unwrapping for NPT
Multi-component support	Partial g(r), directed CN, per-element filters throughout
Performance	Rayon data parallelism; linked-cell neighbour lists for O(N) searches
Extensibility	Strict layered crate architecture; no cross-dependencies between middle layers

Architecture

```
ferro-cli / ferro-python        ← only layer combining multiple crates
      ferro-core                ← Atom, Frame, Trajectory, Cell; static data;
↪  units; errors
      ferro-io        → core    ← format readers / writers
```

```
ferro-structure → core    ← supercell, vacuum, merge, box estimation
ferro-analysis  → core    ← md/, dft/ (Bader, ChgSDF), ml/ (future)
ferro-workflow  → core    ← QC input builders (Gaussian / CP2K / QE)
```

Internal Units

Quantity	Unit
Length	Å
Energy	eV
Force	eV/Å
Stress	eV/Å^3
Time	fs
Mass	amu
Charge	e
Temperature	K

Installation

Requirements

- Rust 1.75+ (`rustup` recommended)
- Cargo (bundled with Rust)

Build from Source

```
git clone https://github.com/ferro/ferro
cd ferro
cargo build --release
```

The compiled CLI binaries are placed in `target/release/`:

Binary	Purpose
`fe-convert`	Format conversion
`fe-info`	Print structure information
`fe-job`	Generate QC input files
`fe-traj`	Structural analysis (g(r), S(q), MSD, angle)
`fe-corr`	Correlation functions (VACF, rotcorr, van Hove)
`fe-cube`	3-D spatial maps (density, jump, radius, SDF)
`fe-network`	Glass network analysis (Qn, CN, ligand classification)

Python Bindings

`ferro-python` is its own workspace and is built with maturin (not the root `cargo build`). `maturin develop` needs an active virtualenv / conda env:

```
pip install maturin
cd ferro-python
maturin develop                                    # into the active env
# or, without an active env:
maturin build --release --interpreter "$(which python)"
pip install target/wheels/ferro-*.whl
```

See Python Bindings for the full API reference.

Generating This Documentation

```
cd docs
mdbook build          # HTML output → docs/book/
mdbook-pandoc         # PDF output → docs/book/ferro.pdf
```

Requires mdbook, mdbook-pandoc, pandoc, and a LaTeX distribution (XeLaTeX recommended).

Part I

Core Data Model

Chapter 1

Core Data Model

Type Hierarchy

Every ferro API accepts and returns `Trajectory`, even for single-frame files.

```
Trajectory
    Vec<Frame>
        Vec<Atom>          - atomic species, positions, optional properties
        Option<Cell>       - None = non-periodic
        [bool; 3]          - pbc flags per axis
        Optional results   - energy, forces, stress, velocities
```

Atom

```rust
pub struct Atom {
    pub element: String,
    pub position: Vector3<f64>,  // Å, Cartesian
    pub label: Option<String>,   // e.g. "Fe1", "Fe2"
    pub mass: Option<f64>,       // None → look up from element table
    pub magmom: Option<f64>,     // initial magnetic moment (DFT input)
    pub charge: Option<f64>,     // Bader / DDEC charge (post-processing)
}
```

The atom **index is implicit** (position in `Vec<Atom>`); no `index` field is stored to prevent inconsistency.

Frame

```rust
pub struct Frame {
    pub atoms: Vec<Atom>,
    pub cell: Option<Cell>,              // None = non-periodic
    pub pbc: [bool; 3],
    pub charge: i32,
    pub multiplicity: u32,
    pub bonds: Option<Vec<(usize,usize)>>,
```

```
    pub energy: Option<f64>,              // eV
    pub forces: Option<Vec<Vector3<f64>>>, // eV/Å
    pub stress: Option<Matrix3<f64>>,     // eV/Å³
    pub velocities: Option<Vec<Vector3<f64>>>, // Å/fs (internal standard)
}
```

`pbc = [false,false,false]` → molecular system.
`pbc = [true,true,false]` → surface / slab.
`pbc = [true,true,true]` → bulk crystal or glass.

Cell

```
pub struct Cell {
    pub matrix: Matrix3<f64>,  // row vectors a, b, c  [Å]
}
```

Method	Description
`from_lengths_angles(a,b,c, , , )`	Construct from lattice parameters
`lengths() -> [f64; 3]`	|a|, |b|, |c|
`angles() -> [f64; 3]`	, , in degrees
`volume() -> f64`	Cell volume [Å³]
`fractional_to_cartesian(f)`	f · M
`cartesian_to_fractional(c)`	c · M [1]
`wrap_position(c)`	Fold Cartesian position into [0, L)
`minimum_image(v)`	Apply minimum-image convention to vector v

Coordinate Conventions

The cell matrix stores row vectors:

$$\mathbf{M} = \begin{pmatrix} \mathbf{a} \\ \mathbf{b} \\ \mathbf{c} \end{pmatrix}$$

Fractional → Cartesian: $\mathbf{r} = \mathbf{f} \cdot \mathbf{M}$

Cartesian → Fractional: $\mathbf{f} = \mathbf{r} \cdot \mathbf{M}^{-1}$

For a triclinic cell:

$$\mathbf{M} = \begin{pmatrix} a & 0 & 0 \\ b\cos\gamma & b\sin\gamma & 0 \\ c\cos\beta & c(\cos\alpha - \cos\beta\cos\gamma)/\sin\gamma & c\sqrt{1-\cos^2\alpha-\cos^2\beta-\cos^2\gamma+2\cos\alpha\cos} \end{pmatrix}$$

Trajectory

```
pub struct Trajectory {
    pub frames: Vec<Frame>,
    pub metadata: Option<TrajectoryMetadata>,
}
```

NPT trajectories (variable box per frame) are handled naturally: each `Frame` carries its own `Cell`.

Pseudo-Element Labels

ferro supports pseudo-element labels for sub-classified atoms (e.g. after network analysis):

Label	Meaning
`P0/P1/P2/P3`	Phosphorus with Qn connectivity
`Of`	Free oxygen (0 P neighbours)
`On`	Non-bridging oxygen (1 P neighbour)
`Ob`	Bridging oxygen (2 P neighbours)
`Zn`, `Na`, …	Modifier cation symbols

Atomic-number lookup (`elem_z`) resolves pseudo-labels by stripping suffixes: `"P3"` → P (Z=15), `"Ob"` → O (Z=8).

Part II

Analysis Methods

Chapter 2

Radial Distribution Function g(r)

Theory

The radial distribution function (RDF) $g_{\alpha\beta}(r)$ describes the probability of finding a particle of type β at distance r from a particle of type α, relative to an ideal gas at the same number density.

Partial g(r)

For a system with N_α atoms of type α and N_β atoms of type β in a volume V:

$$g_{\alpha\beta}(r) = \frac{V}{N_\alpha N_\beta} \frac{\langle n_{\alpha\beta}(r, r+\Delta r) \rangle}{4\pi r^2 \Delta r}$$

where $n_{\alpha\beta}(r, r+\Delta r)$ is the number of β atoms in the shell $[r, r+\Delta r)$ around any α atom.

In practice, the histogram count $C_{\alpha\beta}$ is accumulated and normalised as follows:

Same species ($\alpha = \beta$):

$$g_{\alpha\alpha}(r_i) = \frac{2, C_{\alpha\alpha}(r_i)}{4\pi r_i^2 \Delta r \cdot \frac{N_\alpha - 1}{V} \cdot N_\alpha \cdot N_{\text{frames}}}$$

Different species ($\alpha \neq \beta$):

$$g_{\alpha\beta}(r_i) = \frac{C_{\alpha\beta}(r_i)}{4\pi r_i^2 \Delta r \cdot \frac{N_\beta}{V} \cdot N_\alpha \cdot N_{\text{frames}}}$$

The bin centre is $r_i = r_\text{min} + (i + 0.5) \Delta r$.

Total g(r)

All atoms are treated as a single species:

$$g_{\text{total}}(r_i) = \frac{2, C_{\text{total}}(r_i)}{4\pi r_i^2 \Delta r \cdot \frac{N-1}{V} \cdot N \cdot N_{\text{frames}}}$$

Coordination Number CN(r)

The cumulative coordination number gives the average number of neighbours within distance r.

Directed CN (`"A-B"` = average B atoms within r around each A):

$$\text{CN}_{A \to B}(r) = \sum_{r_i \leq r} \frac{m \cdot C_{AB}(r_i)}{N_A \cdot N_{\text{frames}}}$$

where $m = 2$ for same-species pairs and $m = 1$ for cross-species pairs.
For $A \neq B$: the reverse direction `"B-A"` is also computed with N_B as the denominator.

Minimum-Image Convention

All pair distances use the minimum-image convention:

$$\mathbf{r}_{ij}^{\text{min}} = \mathbf{r}_{ij} - \mathbf{M} \cdot \text{round}!\left(\mathbf{M}^{-1} \mathbf{r}_{ij}\right)$$

This is correct for orthorhombic and triclinic cells as long as $r_{\text{max}} < L_{\text{min}}/2$.

Parameters

```rust
pub struct GrParams {
    pub r_min: f64,   // default: 0.005 Å
    pub r_max: f64,   // default: 10.005 Å  (use with_auto_rmax for safety)
    pub dr: f64,      // default: 0.01 Å
    pub r_cut: f64,   // default: 2.3 Å  (for short-range bond statistics)
}
```

`r_max` must satisfy $r_{\text{max}} < L_{\text{min}}/2$. Use `GrParams::with_auto_rmax(&traj)` to set it automatically from the first frame.

Output

File	Content
`.gr`	Columns: `r[Å]`, then symmetric partial g(r) in periodic-table order, `total` last
`.cn`	Columns: `r[Å]`, then directed CN pairs in (Z_center, Z_neighbor) order

Header lines record all parameters, atom counts, average volume, and number density.

Bond Statistics

Within `r_cut`, the mean and standard deviation of pair distances are computed and printed in the `.cn` header:

$$\bar{d} = \frac{1}{N_{\text{bonds}}} \sum_k d_k, \quad \sigma = \sqrt{\frac{1}{N_{\text{bonds}}} \sum_k (d_k - \bar{d})^2}$$

Usage

```
fe-traj -m gr -i traj.dump -o output
# writes output.gr, output.cn
```

```
use ferro_analysis::md::{GrParams, calc_gr, write_gr, write_cn};

let params = GrParams::with_auto_rmax(&traj);
let result = calc_gr(&traj, &params).unwrap();
write_gr(&result, "output.gr").unwrap();
write_cn(&result, "output.cn").unwrap();
```

Implementation Notes

- Parallelism: per-frame `par_iter` with `fold`/`reduce` accumulation.
- Pseudo-element labels (e.g. `"P0"`, `"Ob"`) are supported: `elem_z` resolves them by stripping numeric/alphabetic suffixes.
- Column ordering uses atomic number Z as the primary sort key, with string labels as tiebreaker for deterministic ordering of pseudo-elements.

Chapter 3

Structure Factor S(q)

Theory

The static structure factor $S(q)$ is computed from the radial distribution function $g(r)$ via a Fourier sine transform (Faber-Ziman formalism).

Partial S(q)

$$S_{\alpha\beta}(q) = 1 + \frac{4\pi\rho}{q} \int_0^\infty r \left[g_{\alpha\beta}(r) - 1 \right] \sin(qr), dr$$

In discretised form:

$$S_{\alpha\beta}(q) = 1 + \frac{4\pi\rho}{q} \sum_{r_i} r_i \left[g_{\alpha\beta}(r_i) - 1 \right] \sin(qr_i), \Delta r$$

where $\rho = N/V$ is the total number density.

Weighted Total S(q)

For experimental comparison, the total S(q) can be weighted by scattering factors.

Faber-Ziman weights:

$$S_{\text{weighted}}(q) = \sum_{\alpha \leq \beta} w_{\alpha\beta}(q) \cdot S_{\alpha\beta}(q)$$

$$w_{\alpha\beta}(q) = \frac{(2 - \delta_{\alpha\beta}), c_\alpha c_\beta f_\alpha(q) f_\beta(q)}{\left[\sum_\gamma c_\gamma f_\gamma(q) \right]^2}$$

where $c_\alpha = N_\alpha / N$ is the mole fraction of species α, and $f_\alpha(q)$ is the scattering factor.

- **X-ray** (`Xrd`): $f_\alpha(q)$ are the q-dependent atomic form factors.

- **Neutron** (`Neutron`): f_α is replaced by the q-independent coherent scattering length b_α^coh.

Physical Interpretation

$S(q)$ characterises structural correlations at length scale $2\pi/q$. Key features:

Feature	Interpretation
First sharp diffraction peak (FSDP) at $q \approx 1$–2 Å $^{-1}$	Medium-range order (network periodicity ~3–5 Å)
Principal peak at $q \approx 2$–3 Å $^{-1}$	Nearest-neighbour distance
$S(q) \to 1$ as $q \to \infty$	Loss of structural correlations at short wavelengths

Parameters

```rust
pub struct SqParams {
    pub q_min: f64,              // default: 0.1 Å ⁻¹
    pub q_max: f64,              // default: 25.0 Å ⁻¹
    pub dq: f64,                 // default: 0.05 Å ⁻¹
    pub weighting: SqWeighting,  // default: None
}

pub enum SqWeighting {
    None,     // equal weights (Faber-Ziman, no form factors)
    Xrd,      // X-ray atomic form factors
    Neutron,  // neutron coherent scattering lengths
    Both,     // compute both simultaneously
}
```

Output

```
fe-traj -m sq -i traj.dump -o output
# writes output.gr, output.sq
```

The `.sq` file header records both the g(r) parameters (used as input) and the S(q) parameters. Column ordering matches the `.gr` file. Additional columns `total_xrd` and/or `total_neutron` are appended when weighting is requested.

Usage

```rust
use ferro_analysis::md::{GrParams, SqParams, SqWeighting, calc_gr,
 ↪  calc_sq_from_gr, write_sq};

let gr = calc_gr(&traj, &GrParams::with_auto_rmax(&traj)).unwrap();
let sq = calc_sq_from_gr(&gr, &SqParams {
    q_min: 0.5, q_max: 20.0, dq: 0.02,
```

```
    weighting: SqWeighting::Neutron,
});
write_sq(&gr, &sq, "output.sq").unwrap();
```

Implementation Notes

- The Fourier transform is parallelised over q values with `rayon::par_iter`.
- Scattering data (form factors and neutron lengths) are stored as static tables in `ferro_analysis::md::scattering_data`.
- For a single-element system, `total_xrd` `total` because all Faber-Ziman weights reduce to 1.

Chapter 4

Mean Squared Displacement (MSD)

Theory

The mean squared displacement (MSD) quantifies how far atoms diffuse over time. In the diffusive regime it grows linearly, and the self-diffusion coefficient D is extracted via the Einstein relation.

Einstein Relation

$$\mathrm{MSD}(t) = \langle |\mathbf{r}(t_0 + t) - \mathbf{r}(t_0)|^2 \rangle$$

$$D = \lim_{t \to \infty} \frac{\mathrm{MSD}(t)}{6t}$$

(factor of 6 for 3-D isotropic diffusion; use 2 for 1-D or 4 for 2-D).

Time-Shift Averaging

To reduce statistical noise, the MSD is averaged over all possible time origins t_0 separated by `shift` frames:

$$\mathrm{MSD}(\tau) = \frac{1}{N_{\text{origins}}} \sum_{p} \frac{1}{N_{\text{atoms}}} \sum_{j} \left| \mathbf{r}_j(p + \tau) - \mathbf{r}_j(p) \right|^2$$

where p runs over ${0, \text{shift}, 2\cdot\text{shift}, \ldots}$ subject to $p + \tau \leq N_\text{frames}$.

Periodic Boundary Handling

Atoms crossing periodic boundaries must be **unwrapped** to obtain true displacements.

Algorithm (matching code1/msd.c `EstimateMSD`):

1. Convert Cartesian coordinates to fractional: $\mathbf{f}_j^{(t)} = \mathbf{r}_j^{(t)} \cdot \mathbf{M}^{-1}$

2. Unwrap: for each step $t > 0$ and each fractional component k:

$$f_{j,k}^{(t)} \leftarrow f_{j,k}^{(t)} - \text{round}!\left(f_{j,k}^{(t)} - f_{j,k}^{(t-1)}\right)$$

3. Compute unwrapped Cartesian displacement $\Delta\mathbf{r}_j = \mathbf{\Delta f}_j \cdot \overline{\mathbf{M}}$

NPT Trajectories

For variable-box (NPT) simulations, the box matrix changes each frame. The total MSD uses the **average** of the origin- and endpoint-frame matrices:

$$\overline{\mathbf{M}} = \frac{\mathbf{M}^{(p)} + \mathbf{M}^{(p+\tau)}}{2}$$

The directional MSD along axis k uses the endpoint cell parameter $L_k^{(p+\tau)}$ (simplified approximation matching code1):

$$\text{MSD}_k(\tau) = \langle(\Delta f_k)^2\rangle \cdot \left(L_k^{(p+\tau)}\right)^2$$

Directional MSD

For periodic systems, `msd_a`, `msd_b`, `msd_c` give displacements along the three crystal axes. For non-periodic systems they correspond to Cartesian x, y, z.

Parameters

```rust
pub struct MsdParams {
    pub tau: Option<usize>,           // window size [frames]; None = all frames
    pub shift: usize,                 // origin spacing [frames]; default: 1
    pub dt: f64,                      // time step [fs]; default: 1.0
    pub elements: Option<Vec<String>>,// None = all atoms
}
```

Output

Columns: `time[fs]`, `msd[Å²]`, `msd_a[Å²]`, `msd_b[Å²]`, `msd_c[Å²]`

Usage

```
fe-traj -m msd -i traj.dump --dt 2.0 --shift 10 -o output.msd
```

```rust
use ferro_analysis::md::{MsdParams, calc_msd, write_msd};

let params = MsdParams { tau: Some(1000), shift: 10, dt: 2.0,
    elements: Some(vec!["Li".into()]) };
let result = calc_msd(&traj, &params).unwrap();
write_msd(&result, "output.msd").unwrap();
```

Extracting the Diffusion Coefficient

Fit the linear region (avoiding the ballistic regime at short t and the noise-dominated long-t tail):

$$D = \frac{1}{6} \cdot \frac{d, \mathrm{MSD}}{d, t}$$

To convert from Å²/fs to cm²/s: multiply by 10^{-16}.

Implementation Notes

- Parallelism: per-origin `par_iter`; `unwrap_frac` runs serially before parallelisation (sequential dependency).
- Non-periodic path: Cartesian coordinates are used directly without unwrapping.

Chapter 5

Bond Angle Distribution

Theory

The bond angle distribution $P(\theta)$ describes the probability of observing a three-body angle A–B–C, where B is the central atom. It is a key structural descriptor for network-forming glasses (e.g. P–O–P, O–P–O angles in phosphate glasses).

Angle Calculation

For a triplet (A, B, C) with B as centre:

$$\theta = \arccos! \left(\frac{\overrightarrow{BA} \cdot \overrightarrow{BC}}{|\overrightarrow{BA}||\overrightarrow{BC}|} \right)$$

The vectors $\overrightarrow{BA}$ and $\overrightarrow{BC}$ use the minimum-image convention for periodic cells.

Selection Criteria

An atom pair (A, B) forms a bond if $|\overrightarrow{BA}| < r_{\text{cut,AB}}$.
Similarly for (B, C): $|\overrightarrow{BC}| < r_{\text{cut,BC}}$.

When both end atoms are the same element: $r_{\text{cut}} = \min(r_{\text{cut,AB}}, r_{\text{cut,BC}})$.

Canonical Key Convention

For each triplet, the key `"ElemA-ElemCenter-ElemC"` is assigned with $Z(\text{ElemA}) \leq Z(\text{ElemC})$. When both end atoms have the same Z, lexicographic order on the label string is used. This ensures each angle type appears exactly once.

No double-counting: the enumeration requires $\text{idx}(A) < \text{idx}(C)$ to avoid counting (A,B,C) and (C,B,A) as separate events.

Statistics

From the raw count histogram $h(\theta_i)$:

$$\bar{\theta} = \frac{\sum_i \theta_i, h_i}{\sum_i h_i}, \quad \sigma = \sqrt{\frac{\sum_i (\theta_i - \bar{\theta})^2 h_i}{\sum_i h_i}}$$

Parameters

```rust
pub struct AngleParams {
    pub r_cut_ab: f64,   // cutoff for lower-Z end to centre [Å]; default: 2.3
    pub r_cut_bc: f64,   // cutoff for higher-Z end to centre [Å]; default: 2.3
    pub d_angle: f64,    // histogram bin width [degrees]; default: 0.1
}
```

Output

Columns: `angle[deg]`, then one column per triplet key in (Z_center, Z_left, Z_right) order.

The header lists mean, standard deviation, and total count per triplet.

Usage

```
fe-traj -m angle -i traj.dump --rcut-ab 2.3 --rcut-bc 2.3 -o output.angle
```

```rust
use ferro_analysis::md::{AngleParams, calc_angle, write_angle};

let params = AngleParams { r_cut_ab: 2.4, r_cut_bc: 2.4, d_angle: 0.5 };
let result = calc_angle(&traj, &params).unwrap();
write_angle(&result, "output.angle").unwrap();
```

Implementation Notes

Linked-Cell List

Brute-force neighbour search is O(N²) per frame. ferro uses a linked-cell list to reduce this to O(N · k) where k is the average number of atoms in the search volume.

For each central atom B, only the 27 cells within ±1 cell in each direction are searched. The cell size is chosen so that a sphere of radius $r_\text{cut,max}$ fits within the search volume:

$$n_k = \left\lfloor \frac{1}{r_{\text{cut,max}} \cdot |(\mathbf{M}^T)^{-1}|_k} \right\rfloor$$

This is exact for triclinic cells.

Parallelism

Computation is parallelised per frame with `rayon::par_iter().fold().reduce()`.

Chapter 6

Van Hove Self-Correlation Function

Theory

The van Hove self-correlation function $G_s(r, \tau)$ is the probability distribution of single-particle displacements over a time interval τ. It provides a more complete picture of atomic motion than the MSD alone: it can reveal non-Gaussian dynamics, heterogeneous mobility, and jump diffusion.

Definition

$$G_s(r, \tau) = \frac{1}{N} \sum_j \langle \delta!\left(r - |\mathbf{r}_j(t_0 + \tau) - \mathbf{r}*j(t_0)|\right) \rangle*{t_0}$$

In practice this is discretised as a histogram $P(r_i, \tau)$ normalised so that $\sum_i P(r_i, \tau) = 1$ (discrete probability mass function).

Gaussian Reference

For a purely diffusive Gaussian process:

$$G_s^{\text{Gaussian}}(r, \tau) = \left(\frac{1}{4\pi D\tau}\right)^{3/2} \exp!\left(-\frac{r^2}{4D\tau}\right) \cdot 4\pi r^2$$

Deviations from this Gaussian form — e.g. a secondary peak at large r — indicate heterogeneous dynamics or discrete jump events.

Non-Gaussian Parameter

The non-Gaussian parameter $\alpha_2(\tau)$ quantifies the deviation from Gaussian behaviour:

$$\alpha_2(\tau) = \frac{3\langle r^4(\tau)\rangle}{5\langle r^2(\tau)\rangle^2} - 1$$

$\alpha_2 = 0$ for a Gaussian distribution; $\alpha_2 > 0$ indicates fat tails (fast-moving particles).

Algorithm

Follows code1/vanhove.c (`EstimateVanHove`):

1. Convert Cartesian positions to fractional coordinates.
2. Unwrap fractional coordinates (same procedure as MSD).
3. Convert back to absolute Cartesian positions.
4. For each time origin p and each selected atom j:

$$r = |\mathbf{r}_j(p+\tau) - \mathbf{r}_j(p)|$$

   Accumulate into histogram bin $\lfloor r / \Delta r \rfloor$.
5. Normalise: $P(r_i) = \text{count}(r_i) / (N_\text{origins} \cdot N_\text{atoms})$.

Parameters

```rust
pub struct VanHoveParams {
    pub tau: Option<usize>,            // lag [frames]; None = n_frames - 1
    pub shift: usize,                  // origin spacing [frames]; default: 1
    pub dt: f64,                       // time step [fs]; default: 1.0
    pub r_min: f64,                    // default: 0.0 Å
    pub r_max: f64,                    // default: 10.0 Å
    pub dr: f64,                       // bin width [Å]; default: 0.01
    pub elements: Option<Vec<String>>, // None = all atoms
}
```

Output

Columns: `r[Å]`, `Gs(r,tau)` (normalised so sum = 1).

The header records the lag time τ in both frames and physical time (fs).

Usage

```
fe-corr -m vanhove -i traj.dump --tau 500 --dt 2.0 -o output.vanhove
```

```rust
use ferro_analysis::md::{VanHoveParams, calc_vanhove, write_vanhove};

let params = VanHoveParams {
    tau: Some(500), shift: 1, dt: 2.0,
    r_min: 0.0, r_max: 8.0, dr: 0.02,
    elements: Some(vec!["Li".into()]),
};
let result = calc_vanhove(&traj, &params).unwrap();
write_vanhove(&result, "output.vanhove").unwrap();
```

Interpreting Results

Feature	Interpretation
Single peak near $r = 0$	Localised, non-diffusing atoms
Broad peak with Gaussian shape	Normal diffusion
Bimodal distribution	Coexistence of slow and fast populations
Secondary peak at $r \approx$ jump length	Discrete jump mechanism

Implementation Notes

- Parallelism: per-origin `par_iter`.
- For periodic systems, fractional-coordinate unwrapping (shared with MSD) is applied before computing displacements.
- For non-periodic systems, raw Cartesian distances are used directly.

Chapter 7

Velocity Autocorrelation Function (VACF)

Theory

The velocity autocorrelation function (VACF) $C_v(t)$ measures how quickly atomic velocities decorrelate over time. Its Fourier transform gives the vibrational density of states (VDOS), which encodes information about lattice dynamics and diffusion mechanisms.

Definition

$$C_v(t) = \langle \mathbf{v}(t_0) \cdot \mathbf{v}(t_0 + t) \rangle = \frac{1}{N} \sum_j \langle \mathbf{v}_j(t_0) \cdot \mathbf{v}*j(t_0 + t) \rangle*{t_0}$$

Expanding into Cartesian components:

$$C_v(t) = \frac{1}{N} \sum_j \left[v_{x,j}(0) v_{x,j}(t) + v_{y,j}(0) v_{y,j}(t) + v_{z,j}(0) v_{z,j}(t) \right]$$

At $t = 0$: $C_v(0) = \langle v^2 \rangle = 3 k_B T / m$ (equipartition theorem).

Self-Diffusion Coefficient

The Green-Kubo relation connects the VACF to the self-diffusion coefficient D:

$$D = \frac{1}{3} \int_0^\infty C_v(t), dt$$

In practice, the running integral is computed as:

$$D(t) = \frac{1}{3} \sum_{i=0}^{n} C_v(i \cdot \Delta t) \cdot \Delta t$$

which converges to D as $t \to \infty$ (rectangular approximation, same as code1).

Vibrational Density of States

The VDOS $g(\nu)$ is obtained via Fourier transform of the VACF:

$$g(\nu) \propto \int_0^\infty C_v(t) \cos(2\pi\nu t), dt$$

Peaks in $g(\nu)$ correspond to characteristic vibrational modes.

Time-Shift Averaging

$$C_v(\tau) = \frac{1}{N_{\text{origins}}} \sum_p \frac{1}{N_{\text{atoms}}} \sum_j \mathbf{v}_j(p) \cdot \mathbf{v}_j(p+\tau)$$

Unlike position-based analyses, **no coordinate unwrapping is needed** — velocities do not have periodic boundary jumps.

Parameters

```
pub struct VacfParams {
    pub tau: Option<usize>,            // window [frames]; None = all frames
    pub shift: usize,                  // origin spacing [frames]; default: 1
    pub dt: f64,                       // time step [fs]; default: 1.0
    pub elements: Option<Vec<String>>, // None = all atoms
}
```

Output

Columns: `time[fs]`, `vacf[v²]`, `vacf_x[v²]`, `vacf_y[v²]`, `vacf_z[v²]`, `diffusion[v²·fs]`

Unit Note

Velocities are stored in whatever unit the source trajectory uses. LAMMPS metal-unit dump files store velocities in Å/ps. To convert to internal standard units (Å/fs), multiply by 10^{-3}. As a consequence:

- C_v in metal units: (Å/ps)² → multiply by 10^{-6} to get (Å/fs)²
- D in metal units: Å²/ps → multiply by 10^{-3} to get Å²/fs → multiply by 10^{-16} to get cm²/s

This conversion will be applied automatically in a future IO unit normalisation update.

Usage

```
fe-corr -m vacf -i traj.dump --dt 2.0 --elements Li -o output.vacf
```

```
use ferro_analysis::md::{VacfParams, calc_vacf, write_vacf};

let params = VacfParams { tau: Some(500), dt: 2.0, shift: 1,
```

```
    elements: Some(vec!["Li".into()]) };
let result = calc_vacf(&traj, &params).unwrap();
write_vacf(&result, "output.vacf").unwrap();
```

Implementation Notes

- Requires `frame.velocities` to be populated (returns `None` if any frame lacks velocities).
- Parallelism: per-origin `par_iter`.

Chapter 8

Rotational Autocorrelation Function C (t)

Theory

The second-order rotational autocorrelation function $C_2(t)$ measures how quickly molecular orientations decorrelate over time. It is defined using the second-order Legendre polynomial P_2:

$$C_2(t) = \langle P_2(\cos\theta(t)) \rangle = \left\langle \frac{3\cos^2\theta(t) - 1}{2} \right\rangle$$

where $\theta(t)$ is the angle between the orientation vector at time t_0 and time $t_0 + t$.

Orientation Vector

For each centre atom c, the orientation vector $\mathbf{u}*c(t)$ is the vectorial sum of all bond vectors to neighbour atoms within the cutoff radius r_cut:

$$\mathbf{u}*c(t) = \sum_{n \in \text{neighbours}(c,, r_\text{cut})} (\mathbf{r}_n - \mathbf{r}*c)_\text{min-image}$$

This generalises to arbitrary A-centre-B structures (e.g. water: O as centre, H as neighbour; phosphate tetrahedra: P as centre, O as neighbour).

P Calculation

For each centre atom j and time origin p:

$$P_2!\left[\cos\theta_j(p,\tau)\right] = \frac{3(\mathbf{u}_j(p)\cdot\mathbf{u}_j(p+\tau))^2}{2|\mathbf{u}_j(p)|^2|\mathbf{u}_j(p+\tau)|^2} - \frac{1}{2}$$

Atoms with zero orientation vector (no neighbours) are excluded.

Time-Shift Averaging

$$C_2(\tau) = \frac{1}{N_{\text{origins}}} \sum_p \frac{1}{N_{\text{mol}}} \sum_j P_2!\left[\cos\theta_j(p,\tau)\right]$$

Physical Properties

Property	Value
$C_2(0)$	Always = 1 (angle with itself is 0°)
$C_2(\infty)$	→ 0 for freely rotating molecules
Range	$[-0.5,\ 1]$

$C_2(t)$ for a perfect rotor decays exponentially: $C_2(t) = e^{-t/\tau_c}$, where τ_c is the rotational correlation time.

Rotational Correlation Time

The rotational correlation time τ_c is obtained from the running integral:

$$\tau_c(t) = \int_0^t C_2(t'), dt'$$

which converges to τ_c as $t \to \infty$.

Parameters

```rust
pub struct RotCorrParams {
    pub center: String,       // centre element, e.g. "O"
    pub neighbor: String,     // neighbour element, e.g. "H"
    pub r_cut: f64,           // cutoff radius [Å]; default: 1.2
    pub tau: Option<usize>,   // window [frames]; None = all frames
    pub shift: usize,         // origin spacing; default: 1
    pub dt: f64,              // time step [fs]; default: 1.0
}
```

Output

Columns: `time[fs]`, `C(t)`, `integral[fs]`

Usage

```
fe-corr -m rotcorr -i traj.dump --center P --neighbor O --rcut 2.4 --dt 2.0 -o
 ↪  output.rotcorr
```

```rust
use ferro_analysis::md::{RotCorrParams, calc_rotcorr, write_rotcorr};

let params = RotCorrParams {
    center: "P".into(), neighbor: "O".into(),
    r_cut: 2.4, tau: Some(500), shift: 5, dt: 2.0,
};
let result = calc_rotcorr(&traj, &params).unwrap();
write_rotcorr(&result, "output.rotcorr").unwrap();
```

Implementation Notes

- Orientation vectors are precomputed for all frames before the parallel loop.
- For periodic systems, the minimum-image convention is applied to bond vectors.
- Parallelism: per-origin `par_iter`.

Chapter 9

3-D Spatial Density Maps

Theory

The spatial density map divides the simulation box into an $n_x \times n_y \times n_z$ voxel grid and accumulates a time-averaged scalar quantity for each voxel. Three modes are supported:

Mode	Accumulated quantity	Unit
`Density`	atomic number density	atoms/Å^3
`Velocity`	mean atomic speed $	\mathbf{v}
`Force`	mean atomic force magnitude $	\mathbf{f}

The result is a Gaussian cube file that can be visualised directly in VESTA, VMD, or similar tools.

Density Mode

Each atom is mapped to a voxel by converting its Cartesian position to fractional coordinates:

$$\mathbf{f}*j* = *mathbf{M}^{-1}* *mathbf{r}*j, \quad f*{j,k}* = *f*{j,k} \bmod 1$$

The voxel index along axis k is:

$$i_k = \lfloor f_{j,k} \cdot n_k \rfloor \bmod n_k$$

After accumulating counts over all atoms and frames, the number density in voxel (i, j, k) is:

$$\rho_{ijk} = \frac{\text{count}*{ijk}*}{*N*\text{frames} \cdot V_\text{voxel}}$$

where the voxel volume is:

$$V_\text{voxel} = \frac{V_\text{cell}}{n_x \cdot n_y \cdot n_z}$$

Velocity / Force Mode

For `Velocity` and `Force` modes the average magnitude is computed per voxel:

$$\langle |\mathbf{q}| \rangle_{ijk} = \frac{\sum_{\text{visits}} |\mathbf{q}|}{\text{count}_{ijk}}$$

where $\mathbf{q}$ is the velocity or force vector of the visiting atom. Voxels with no visits are set to zero.

Voxel Spacing Matrix

The output cube file encodes the voxel step vectors. For a general triclinic cell with matrix $\mathbf{M}$ (rows = $\mathbf{a}, \mathbf{b}, \mathbf{c}$), the spacing matrix is:

$$\mathbf{S} = \left(\mathbf{a}/n_x \;\; \mathbf{b}/n_y \;\; \mathbf{c}/n_z \right)$$

This ensures correct spatial mapping for both orthogonal and triclinic simulation cells.

Time-Averaged Structure

The cube file header includes a time-averaged atomic structure (mean position over all frames), which serves as a reference geometry for visualisation.

Parameters

```
pub struct CubeDensityParams {
    pub nx: usize,                          // grid divisions along a axis; default:
     ↪ 50
    pub ny: usize,                          // grid divisions along b axis; default:
     ↪ 50
    pub nz: usize,                          // grid divisions along c axis; default:
     ↪ 50
    pub elements: Option<Vec<String>>,      // None = all atoms
    pub mode: CubeMode,                     // Density | Velocity | Force; default:
     ↪ Density
}
```

Output

A Gaussian cube file (`.cube`):

- Header: time-averaged atomic positions + unit cell
- Data: $n_x \times n_y \times n_z$ scalar field

The cube format is directly accepted by VESTA, VMD, Ovito, and most electronic-structure visualisation packages.

Usage

```
# Number density of all atoms
fe-cube -m density -i traj.dump --nx 80 --ny 80 --nz 80 -o density.cube

# Li-only density
fe-cube -m density -i traj.dump --elements Li -o li_density.cube

# Time-averaged velocity magnitude
fe-cube -m velocity -i traj.dump --metal-units -o velocity.cube

# Time-averaged force magnitude
fe-cube -m force -i traj.dump -o force.cube
```

```
use ferro_analysis::md::{CubeDensityParams, CubeMode, calc_cube_density};
use ferro_io::write_cube;

let params = CubeDensityParams {
    nx: 80, ny: 80, nz: 80,
    elements: Some(vec!["Li".into()]),
    mode: CubeMode::Density,
};
let result = calc_cube_density(&traj, &params).unwrap();
write_cube("li_density.cube", &result.cube).unwrap();
```

Implementation Notes

- Parallelism: per-frame `par_iter`. Each frame independently produces `(count, value_sum)` arrays; results are reduced by element-wise addition.
- Fractional coordinates are folded with `rem_euclid(1.0)` to handle atoms outside the nominal box (e.g. from NPT fluctuations or wrapped dump files).
- `Velocity` mode requires `frame.velocities` to be populated; `Force` mode requires `frame.forces`. Frames missing the required data are silently skipped.
- For NPT trajectories the spacing matrix is taken from the first periodic frame; the density is therefore referenced to that cell geometry.

Chapter 10

Jump Distance Distribution

Theory

The jump distribution map identifies spatial hotspots of large atomic displacements (jump events). For each atom and each time window $[t„ t+\tau]$, the true Cartesian displacement is computed from unwrapped fractional coordinates. If the displacement exceeds a threshold d_min, the event is recorded at a voxel position on the 3-D grid.

This analysis is particularly useful for characterising ion hopping in glassy or disordered materials: the resulting cube shows *where* jumps originate, terminate, or pass through, enabling the identification of preferred migration corridors.

Fractional-Coordinate Unwrapping

To obtain the minimum-image displacement correctly for periodic (and NPT) trajectories, fractional coordinates are unwrapped in-place before computing differences. For a single atom over all frames:

$$f_{j,k}^{(t)} \leftarrow f_{j,k}^{(t)} - \text{round!}\left(f_{j,k}^{(t)} - f_{j,k}^{(t-1)}\right), \quad k \in a, b, c$$

This step-by-step correction removes periodic-boundary crossings from the fractional trajectory.

Displacement Calculation

For each time window $[t„ t+\tau]$, the unwrapped fractional displacement is:

$$\Delta\mathbf{f} = \mathbf{f}^{(t+\tau)} - \mathbf{f}^{(t)}$$

The true Cartesian displacement uses the average cell matrix of the two endpoint frames (consistent with the MSD NPT treatment):

$$\Delta\mathbf{r} = \overline{\mathbf{M}}^{T}, \Delta\mathbf{f}, \quad \overline{\mathbf{M}} = \frac{\mathbf{M}^{(t)} + \mathbf{M}^{(t+\tau)}}{2}$$

A jump event is recorded when $|\Delta\mathbf{r}| > d_\text{min}$.

Recording Position

Each jump event is recorded at the atom's wrapped fractional position. Three choices are available:

Option	Recorded position	Typical use
`Start`	$\mathbf{f}^{(t)}$ wrapped	Identify jump origins
`End`	$\mathbf{f}^{(t+\tau)}$ wrapped	Identify jump destinations
`Midpoint`	$(\mathbf{f}^{(t)} + \mathbf{f}^{(t+\tau)})/2$ wrapped	Highlight migration pathways

The voxel index is computed by folding the fractional coordinate into $[0, 1)$ and multiplying by the grid size.

Grid Value

Each voxel stores the raw count of jump events recorded at that location. No normalisation is applied; to convert to a probability density, divide by the total event count.

Parameters

```
pub struct CubeJumpParams {
    pub nx: usize,                          // grid divisions along a axis; default:
     ↪ 50
    pub ny: usize,                          // grid divisions along b axis; default:
     ↪ 50
    pub nz: usize,                          // grid divisions along c axis; default:
     ↪ 50
    pub tau: usize,                         // displacement lag [frames]; default: 1
    pub threshold: f64,                     // minimum displacement [Å]; default:
     ↪ 1.0
    pub elements: Option<Vec<String>>,      // None = all atoms
    pub record_at: JumpPosition,            // Start | End | Midpoint; default:
     ↪ Start
}
```

Output

A Gaussian cube file with raw jump-event counts per voxel. Typical post-processing:

1. Load in VESTA with isosurface rendering to visualise migration channels.
2. Normalise by total jump count to obtain the spatial probability distribution.

Usage

```
# Jump origins for Li, lag = 1 frame, threshold = 1.0 Å
```

```
fe-cube -m jump -i traj.dump --elements Li --tau 1 --threshold 1.0 -o jump.cube

# Jump midpoints (migration pathway visualisation)
fe-cube -m jump -i traj.dump --elements Li --record-at midpoint -o jump_mid.cube
```

```
use ferro_analysis::md::{CubeJumpParams, JumpPosition, calc_cube_jump};
use ferro_io::write_cube;

let params = CubeJumpParams {
    nx: 50, ny: 50, nz: 50,
    tau: 1,
    threshold: 1.0,
    elements: Some(vec!["Li".into()]),
    record_at: JumpPosition::Start,
};
let result = calc_cube_jump(&traj, &params).unwrap();
write_cube("jump.cube", &result.cube).unwrap();
println!("{} jump events recorded", result.n_jumps);
```

Implementation Notes

- Parallelism: per-atom `par_iter`. Each atom independently unwraps its fractional trajectory and accumulates jump events into a local grid; results are reduced by element-wise addition.
- Fractional coordinates for all frames are pre-extracted into a `Vec<Vec<[f64;3]>>` before parallelisation (shared read-only, zero copying per thread).
- For NPT trajectories, cell matrices are also cached per frame so that the average $\overline{\mathbf{M}}$ can be computed without locking.
- Atoms absent from some frames (length-mismatch guard) are silently skipped.

Chapter 11

Hard-Sphere Occupancy

Theory

The hard-sphere occupancy map treats each atom as a sphere of radius r. For each (frame, atom) pair, every voxel whose centre lies within r of the atom is incremented by 1. The resulting 3-D grid represents the cumulative time-averaged occupancy: high values indicate regions of space that are frequently occupied by the selected atom type.

Unlike the density mode (which bins atoms by position), this method smears each atom over a finite volume, producing smoother maps suitable for visualising preferred coordination sites or channel geometry in disordered materials.

Voxel Inclusion Criterion

For an atom at fractional coordinate $\mathbf{f}_\text{atom}$ and a voxel with centre at fractional coordinate $\mathbf{v}$:

1. Compute the fractional displacement with minimum-image convention:

$$\Delta\mathbf{f} = \mathbf{f}_\text{atom} - \mathbf{v}, \quad \Delta f_k \leftarrow \Delta f_k - \text{round}(\Delta f_k)$$

2. Convert to Cartesian using the cell matrix transpose:

$$\Delta\mathbf{r} = \mathbf{M}^T \Delta\mathbf{f}$$

3. Increment the voxel count if $|\Delta\mathbf{r}| < r$.

The voxel centre fractional coordinates are:

$$v_k = \frac{i_k + 0.5}{n_k}, \quad i_k \in 0, \ldots, n_k - 1$$

Bounding-Box Optimisation

Brute-force comparison of every atom against every voxel is $O(N \cdot n_x n_y n_z)$. Instead, for each atom, only the voxels within a bounding box of half-width

$$s_k = \lceil r \cdot \left| (\mathbf{M}^T)^{-1}_k \right| \cdot n_k \rceil + 1$$

around the atom's home voxel are checked. The row norm of $(\mathbf{M}^T)^{-1}$ gives the maximum fractional offset in direction k corresponding to a Cartesian distance of r, making the bound exact for triclinic cells.

Parameters

```rust
pub struct CubeRadiusParams {
    pub nx: usize,                          // grid divisions along a axis; default:
     ↪ 100
    pub ny: usize,                          // grid divisions along b axis; default:
     ↪ 100
    pub nz: usize,                          // grid divisions along c axis; default:
     ↪ 100
    pub radius: f64,                        // hard-sphere radius [Å]; default: 0.7
    pub elements: Option<Vec<String>>,      // None = all atoms
}
```

Output

A Gaussian cube file with raw occupancy counts per voxel. Values are not normalised. To convert to fractional occupancy, divide by $N_\text{frames} \cdot N_\text{atoms}$.

Usage

```
# Li occupancy map, 100³ grid, radius 0.7 Å
fe-cube -m radius -i traj.dump --elements Li --radius 0.7 --nx 100 --ny 100 --nz
 ↪  100 -o li_radius.cube

# All-atom occupancy
fe-cube -m radius -i traj.dump --radius 1.0 -o all_radius.cube
```

```rust
use ferro_analysis::md::{CubeRadiusParams, calc_cube_radius};
use ferro_io::write_cube;

let params = CubeRadiusParams {
    nx: 100, ny: 100, nz: 100,
    radius: 0.7,
    elements: Some(vec!["Li".into()]),
};
let result = calc_cube_radius(&traj, &params).unwrap();
write_cube("li_radius.cube", &result.cube).unwrap();
```

Comparison with Density Mode

Feature	Density mode	Radius mode
Resolution	Depends on bin width $1/n_k$	Depends on r and n_k
Smoothing	None (point-like atoms)	Hard-sphere smearing
Normalisation	atoms/Å^3	Raw counts
Typical use	Structural analysis	Site visualisation

Implementation Notes

- Parallelism: per-frame `par_iter`. Each frame independently produces a count array; results are reduced by element-wise addition.
- The bounding-box optimisation reduces the inner loop from $O(n_x n_y n_z)$ to $O(s_x s_y s_z)$ per atom, giving roughly $10\times$ speedup at default settings.
- Fractional coordinates are folded into $[0, 1)$ with `rem_euclid` before the bounding-box centre is computed, ensuring correct periodic wrapping.

Chapter 12

Cluster SDF (Spatial Distribution Function)

Theory

The cluster SDF identifies all phosphate clusters of a specified Q_n type from an MD trajectory, aligns them to a common reference frame using rigid-body superposition, and accumulates per-atom-type 3-D probability densities. The result reveals the average local geometry and spatial spread of the cluster type in a simulation-cell-independent local coordinate frame.

Q_n Classification

Each network-former atom (e.g. P) is assigned a Q_n label based on the number of bridging ligands (e.g. O) it shares with other former atoms:

Label	Bridging ligands	Description
Q_0	0	Isolated tetrahedron
Q_1	1	Chain end
Q_2	2	Chain middle
Q_3	3	Branching point

Cluster Identification

Connected components of former atoms sharing bridging ligands are found by graph traversal. Each component is classified by the **highest individual Q_n** within it: a component containing one Q_3 and two Q_1 atoms is a Q_3 cluster.

Atom-Type Labels

Each atom in the cluster receives one of the following labels:

Label	Meaning
`P0` / `P1` / `P2` / `P3`	Former atom with its individual Q_n

Label	Meaning
`Of`	Free (non-bridging) oxygen bonded to exactly one former
`On`	Non-bridging oxygen bonded to one former + one modifier
`Ob`	Bridging oxygen shared by two formers
`OBO/Zn`	Modifier cation (if `modifier` is set)

Signature

The composition of atom-type labels is encoded in a canonical signature string, e.g.:

`"Ob:3|On:6|P1:3|P3:1|Zn:2"`

Clusters with the same signature form one **family**; the first cluster in the trajectory serves as the reference structure.

Alignment Strategy

Rigid-body alignment is performed using the **Kabsch algorithm**, which minimises the RMSD between the current cluster and the reference after optimal rotation. To handle atom permutation symmetry, alignment candidates are generated by permuting atoms within groups of the same label:

Cluster type	Permutation set	Max permutations
Q_0 (single P)	All O atoms	$4! = 24$
Q_1/Q_2/Q_3	Same-label P groups	$\leq 3! = 6$

The permutation giving the lowest RMSD is kept.

Kabsch Algorithm

Given two sets of N corresponding points ${P_i}$ (current) and ${Q_i}$ (reference), both centred at the origin:

1. Compute the cross-covariance matrix $\mathbf{H} = \sum_i P_i Q_i^T$.
2. Singular value decomposition: $\mathbf{H} = \mathbf{U} \boldsymbol{\Sigma} \mathbf{V}^T$.
3. Optimal rotation: $\mathbf{R} = \mathbf{V}, \text{diag}(1, 1, \det(\mathbf{V}\mathbf{U}^T)), \mathbf{U}^T$.
4. Apply $\mathbf{R}$ to all atoms in the current cluster.

The determinant correction in step 3 prevents reflection (improper rotation) when the SVD produces a left-handed coordinate system.

Density Accumulation

After alignment, each atom at position $\mathbf{r}$ contributes a Gaussian kernel to the 3-D grid:

$$\rho(\mathbf{x}) \mathrel{+}= \exp!\left(-\frac{|\mathbf{x}-\mathbf{r}|^2}{2\sigma_{\text{px}}^2}\right)$$

where σ_px is the Gaussian broadening width in voxels. All atoms of the same type accumulate into the same grid. Separate grids are maintained for each cluster family (signature group).

RMSD Quality Check

After alignment, the RMSD of the scaffold atoms (P atoms for Q_1/Q_2/Q_3; all O atoms for Q_0) is computed:

$$\text{RMSD} = \sqrt{\frac{1}{N}\sum_i |\mathbf{r}_i^{\text{aligned}} - \mathbf{r}_i^{\text{ref}}|^2}$$

If RMSD exceeds `rmsd_warn_threshold`, a warning is printed to stderr flagging the cluster as a structural outlier.

Parameters

```rust
pub struct ClusterSdfParams {
    pub former: String,              // network-former element, e.g. "P"
    pub ligand: String,              // bridging-ligand element, e.g. "O"
    pub target_qn: u8,               // 0/1/2/3 - cluster Qn level
    pub former_ligand_cutoff: f64,   // former-ligand bond cutoff [Å]; default:
     ↪ 2.4
    pub modifier: Option<String>,    // modifier cation element, e.g. Some("Zn")
    pub modifier_cutoff: f64,        // modifier-ligand cutoff [Å]; default: 2.8
    pub grid_res: f64,               // grid resolution [Å/voxel]; default: 0.1
    pub sigma: f64,                  // Gaussian broadening [voxels]; default:
     ↪ 1.5
    pub padding: f64,                // grid boundary margin [Å]; default: 3.0
    pub rmsd_warn_threshold: f64,    // RMSD warning threshold [Å]; default: 0.5
}
```

Output

One Gaussian cube file per atom type per family:

- Single family: `<stem>_<label>.cube` (e.g. `sdf_P3.cube`, `sdf_Ob.cube`)
- Multiple families: `<stem>_fam0_<label>.cube`, `<stem>_fam1_<label>.cube`, …

All cube files within a family share the same grid origin and spacing, enabling direct overlay in VESTA or VMD.

Usage

```
# Q3 phosphate clusters with Zn modifier
fe-cube -m sdf -i traj.dump --qn 3 --former P --ligand O --cutoff-fl 2.4 \
         --modifier Zn --cutoff-ml 2.8 --grid-res 0.1 --sigma 1.5 -o sdf
```

```
# Q0 isolated tetrahedra (no modifier)
fe-cube -m sdf -i traj.dump --qn 0 --former P --ligand O -o q0_sdf
```

```
use ferro_analysis::md::{ClusterSdfParams, calc_cluster_sdf};
use ferro_io::write_cube;

let params = ClusterSdfParams {
    former: "P".into(),
    ligand: "O".into(),
    target_qn: 3,
    former_ligand_cutoff: 2.4,
    modifier: Some("Zn".into()),
    modifier_cutoff: 2.8,
    grid_res: 0.1,
    sigma: 1.5,
    padding: 3.0,
    rmsd_warn_threshold: 0.5,
};
let result = calc_cluster_sdf(&traj, &params).unwrap();

for (sig, family) in &result.families {
    println!("Family {}: {} clusters", sig, family.n_clusters);
    for (label, cube) in &family.grids {
        write_cube(&format!("sdf_{label}.cube"), cube).unwrap();
    }
}
```

Interpreting Results

Observation	Interpretation
Single family, narrow Gaussian peaks	Highly ordered cluster geometry
Multiple families	Topological isomers (different O connectivity patterns)
High RMSD warnings	Structurally distorted clusters; may warrant separate analysis
Broad `Ob` distribution	Flexible bridging-oxygen geometry
Sharp `P3` peak	Rigid network-former backbone

Implementation Notes

- Cluster identification uses `petgraph` BFS on a bond graph constructed per frame.
- The Kabsch algorithm is applied in the centred frame; the grid origin is positioned at the reference centroid minus `padding`.
- Parallelism: cluster extraction is serial per frame (bond graph is not thread-safe to build in parallel); Gaussian accumulation is serial per cluster (grids are mutably borrowed by family).
- For trajectories with many frames, performance is dominated by bond-graph construction ($O(N^2)$ per frame without a cell list).

Chapter 13

Averaged Charge-Density SDF (`chg-sdf`)

`fe-cube -m chg_sdf` computes the **averaged electron-density spatial distribution function** over a set of Qn-type network clusters. Given a collection of QE `pp.x` charge-density cube files (one per MD snapshot), the tool:

1. Identifies every Qn cluster in each snapshot using the same Kabsch-alignment pipeline as `fe-cube -m sdf`.
2. Extracts a cubic sub-grid of the electron density centred on the cluster anchor atom, interpolating the raw charge-density grid with trilinear interpolation (PBC-safe).
3. Rotates the sub-grid to align with the first-encountered reference cluster via the Kabsch rotation matrix, using "pull" interpolation (`output[r] = input[R`$^{\mathtt{T}}$ `r]`).
4. Accumulates rotated sub-grids; divides by the cluster count to obtain the averaged density.
5. Writes one Gaussian cube file per signature family.

The result reveals the orientationally averaged electron-density distribution around a specific local structural motif (e.g. Q2 PO unit in a phosphate glass).

Prerequisites

Charge-density cube files must be produced by QE `pp.x` (or any program that outputs the Gaussian cube format with atom coordinates in Bohr and density in `e/Bohr`$^{\mathtt{3}}$). One cube file per MD frame is expected.

Usage

```
# Basic: average charge density around Q2 clusters (P-O system)
fe-cube -m chg_sdf \
    --cubes frame_000.cube frame_001.cube frame_002.cube \
    --qn 2 --former P --ligand O --cutoff-fl 2.4 \
```

```
    -o chg_sdf

# With modifier cation (e.g. Zn2+ in Zn-phosphate glass)
fe-cube -m chg_sdf \
    --cubes *.cube \
    --qn 2 --former P --ligand O --cutoff-fl 2.4 \
    --modifier Zn --cutoff-ml 2.8 \
    --chg-padding 6.0 \
    -o chg_sdf_Q2

# Tighter sub-grid / multiple threads
fe-cube -m chg_sdf \
    --cubes *.cube \
    --qn 3 --former P --ligand O \
    --chg-padding 5.0 --rmsd-warn 0.3 \
    --ncore 8 \
    -o q3_density
```

Parameters

Flag	Default	Description
`--cubes <files…>`	(required)	One or more QE pp.x cube files; each file = one MD frame
`--qn`	`3`	Target Qn cluster level (0, 1, 2, or 3)
`--former`	`P`	Network-former element symbol
`--ligand`	`O`	Bridging-ligand element symbol
`--cutoff-fl`	`2.4`	Former–ligand bond cutoff [Å]
`--modifier`	(none)	Modifier cation element; omit if not needed
`--cutoff-ml`	`2.8`	Modifier–ligand cutoff [Å]
`--chg-padding`	`6.0`	Sub-grid margin around the cluster anchor [Å]
`--rmsd-warn`	`0.5`	RMSD threshold for alignment quality warnings [Å]
`--ncore`	all	Parallel threads
`-o`	`chg_sdf`	Output file stem

Note: `--chg-padding` controls the sub-grid size, not the atom SDF grid. Larger values capture more surrounding density but increase memory and runtime.

Output

For a single signature family:

`chg_sdf_Q2.cube`

For multiple families (distinct cluster topologies at the same Qn):

```
chg_sdf_fam0_Q2.cube
chg_sdf_fam1_Q2.cube
...
```

The cube file contains:

- **Atom positions**: the reference cluster atoms in a local Cartesian frame (anchor at origin).
- **Density values**: averaged `_phys × V_cell` in internal units. To convert to physical electron density [e/Å^3], divide by the voxel volume in Å^3.

Console output reports per-family cluster count and alignment RMSD statistics:

```
Family "P-O-O-P"  (142 clusters, RMSD mean=0.031 max=0.187 Å, 0 warnings) →
 ↪  chg_sdf_Q2.cube
ChgSDF Q2 done: 50 frames, 142 clusters total, 1 cube file written
```

Algorithm Details

Cluster identification

Identical to `fe-cube -m sdf`: finds all Qn-type connected components of former atoms sharing bridging ligands, using Union-Find on the former–ligand–former connectivity graph.

Sub-grid extraction

For each cluster anchor position `c` (Cartesian, Å):

`sub[ix, iy, iz] = _charge( c + Δr )`

where `Δr = (ix − half_n, iy − half_n, iz − half_n) × _voxel` and `_charge` is evaluated via trilinear interpolation with PBC folding on the original charge-density grid. The sub-grid has shape `n × n × n` with `n = 2 × padding / _voxel + 1`.

Grid rotation

For each subsequent cluster the Kabsch rotation matrix `R` is computed from the atom positions (same as the atom SDF). The sub-grid is rotated using "pull" interpolation:

`output[r_out] = input[ R ⁻¹ · (r_out − centre) + centre ]`

`R ⁻¹ = R` for a proper rotation. Points outside the sub-grid boundary return 0.

Accumulation

`_avg = (1 / N) Σ  _rotated`

Related Commands

- `fe-cube -m sdf` — atom-type SDF (no charge density required)
- `fe-bader` — Bader charge decomposition from CHGCAR / cube files

Chapter 14

Glass Network Analysis (`fe-network`)

`fe-network` MD

(CN)	
	FO / NBO / BO / OBO
Qn	
	Free / T / B / M

---

cutoff minimum-image convention

$$d_{ij} = \bigl|\mathbf{r}*{ij} - *mathbf{M} *cdot *text{round}!*left(*mathbf{M}^{-1}*mathbf{r}*{ij}\right)\bigr|$$

PBC

(CN)

i α β

$$\text{CN}^{(\alpha\beta)}*i = *bigl/{j : *text{elem}(j)=*beta,; d*{ij} < r_{\alpha\beta}^{\text{cut}}}\bigr|$$

CN CN

CN CN

k β n_k

n_k		
0	`FO`	Free oxygen
1	`NBO(X)`	Non-bridging oxygen X =
2	`BO(X-Y)`	Bridging oxygen X Y
3	`OBO(X-Y-Z)`	Over-bridging oxygen

P + Si `BO(P-Si)` P Si

Qn

i Q_n BO OBO

$$n_i = \left| k \in \text{neighbors}(i) : \text{label}(k) \in \text{BO}, \text{OBO} \right|$$

k i

- `Q3` — 3 P-O-P
- `Q3(2Al)` — 3 2 P-O-Al
- `Q4(1Al,1Si)` — 4 1 P-O-Al 1 P-O-Si

`Q{n}({count}{Elem},...)`.

modifier Zn^{2} Na NBO m NBO p_m

p_m			
0	`Free`	NBO	
1	`T`	Terminal	NBO
2	`B`	Bridging	NBO
3	`M`	Multi 3	NBO

```
# P O   P-O   2.3 Å   CSV
fe-network -i traj.dump --P-O=2.3

#   P-Si
fe-network -i traj.dump --P-O=2.3 --Si-O=1.8 -o result.csv

#   Zn
fe-network -i traj.dump --P-O=2.3 --Zn-O=3.5 --modifier Zn

#
```

```
fe-network -i traj.dump --P-O=2.3 --Zn-O=3.5 --Na-O=3.2 --modifier Zn,Na

#   xlsx  sheet
fe-network -i traj.dump --P-O=2.3 --format xlsx -o result.xlsx

#    500  8
fe-network -i traj.dump --P-O=2.3 --last-n 500 --ncore 8
```

```
fe-network    --Former-Ligand=cutoff
```

```
--P-O=2.3        P    O      2.3 Å
--Si-O=1.8       Si-O   1.8 Å
--Al-O=2.0       Al-O   2.0 Å
--Al-F=2.1       Al-F   2.1 Å
```

`-i <file>`	—	
`-o <file>`	`network`	
`--format csv|xlsx`	`csv`	
`--last-n N`		N
`--ncore N`		
`--metal-units`		LAMMPS metal Å/ps eV/Å
`--modifier Elem`	—	`Zn` `Zn,Na`

```
      --Elem-O=cutoff      --modifier
```

CSV

`<stem>_cn.csv`	(former, ligand)	CN CN
`<stem>_ligand.csv`		FO/NBO/BO/OBO
`<stem>_qn.csv`		Qn
`<stem>_modifier.csv`		`--modifier`

_cn.csv

```
Former,Ligand,CN,Count,Fraction
P,O,3,120,0.240000
P,O,4,380,0.760000
P,O,mean,3.7600,
P,total,4,500,1.000000
P,total,mean,3.7600,
```

_ligand.csv

```
Ligand,Class,Count,Fraction
O,FO,50,0.033333
O,NBO(P),650,0.433333
O,BO(P-P),800,0.533333
```

_qn.csv

```
Former,Species,Count,Fraction
P,Q2,120,0.240000
P,Q3,380,0.760000
```

_modifier.csv

```
Modifier,Role,Count,Fraction
Zn,Free,10,0.040000
Zn,T,120,0.480000
Zn,B,110,0.440000
Zn,M,10,0.040000
```

XLSX

`--format xlsx` sheet

Sheet	
CN	
Ligand	
Qn	Qn
Modifier	

P O	P	O	P-O: 2.3 Å
SiO	Si	O	Si-O: 1.8 Å
Al O	Al	O	Al-O: 2.1 Å

GeO	Ge	O	Ge-O: 2.0 Å
ZnO-P O	P	O	P-O: 2.3 Å Zn modifier : Zn-O: 3.5 Å

g(r) `fe-traj -m gr`

Part III

Input Generation

Chapter 15

Job Builders

`fe-job` generates ready-to-run input files for quantum-chemistry / DFT codes from any structure `ferro` can read (XYZ, CIF, PDB, POSCAR, LAMMPS dump, …).

```
fe-job                                  # overview of supported software
fe-job -s cp2k                          # CP2K-specific help
fe-job -i struct.cif -s qe -o pw.in     # generate a QE input
```

Target	`-s`	value	Output
Gaussian 16/09	`gaussian`	`.gjf`	molecular DFT, method + basis
CP2K (Quickstep)	`cp2k`	`.inp`	periodic DFT/MD; GPW; precise basis DB
Quantum ESPRESSO	`qe`	`.in`	plane-wave `pw.x`; scf/relax/md/bands

All three share charge/spin handling (see Spin Estimation).

Gaussian

```
fe-job -i mol.xyz -s gaussian -m B3LYP -b 6-31G* -o job.gjf
fe-job -i radical.xyz -s gaussian --charge 0 --multiplicity 2
```

`-m` method and `-b` basis set are written verbatim into the route section.

CP2K

A full GPW/Quickstep input: `&GLOBAL`, `&FORCE_EVAL` (`&SUBSYS`, `&DFT`), and a task-specific `&MOTION`/`&VIBRATIONAL_ANALYSIS` block.

```
# Geometry optimisation, PBE-D3(BJ)
```

```
fe-job -i glass.xyz -s cp2k --task geo-opt --functional pbe --dispersion d3bj

# AIMD, 1500 K, CSVR thermostat
fe-job -i glass.xyz -s cp2k --task md --temperature 1500 --md-steps 50000

# Hybrid PBE0 with OT, all-electron pob basis
fe-job -i crystal.cif -s cp2k --functional pbe0 --scf ot --cp2k-basis pob-tzvp
```

Tasks: `energy`, `force`, `geo-opt`, `cell-opt`, `md`, `freq`. Functionals: PBE/BLYP/revPBE/PBEsol (GGA), PBE0/B3LYP/HSE06 (hybrid, auto `&HF` block), SCAN/r²SCAN (meta-GGA via LIBXC).

Precise basis & pseudopotential matching

CP2K basis-set and pseudopotential names are **element- and valence-specific** (e.g. `DZVP-MOLOPT-SR-GTH-q16` for Fe vs `-q6` for O). `ferro` ships a database of **2829 entries** parsed from the official CP2K files (`BASIS_MOLOPT`, `BASIS_MOLOPT_UCL`, `BASIS_MOLOPT_UZH`, `BASIS_pob`, `POTENTIAL`, `POTENTIAL_UZH`), covering:

- **PBE / GGA** MOLOPT basis sets, all elements, all levels
- **SCAN** MOLOPT basis sets
- **All-electron** `pob` basis sets (`pob-dzvp`, `pob-tzvp`)

For each `&KIND` the builder looks up the exact basis name and a pseudopotential whose valence count `q` matches, preferring canonical names (`GTH-PBE` over `GTH-NLCC-PBE`). Unknown/exotic elements fall back to a heuristic q-guess. `--cp2k-basis` selects the family; the per-element exact name is resolved automatically.

Quantum ESPRESSO

A complete `pw.x` input with `ibrav = 0`: `&CONTROL`, `&SYSTEM`, `&ELECTRONS`, plus `&IONS`/`&CELL` for relaxation/MD, then `ATOMIC_SPECIES`, `CELL_PARAMETERS angstrom`, `ATOMIC_POSITIONS angstrom`, `K_POINTS`.

```
# SCF, Gamma point
fe-job -i crystal.cif -s qe

# Metal: Methfessel-Paxton smearing + k-mesh
fe-job -i metal.cif -s qe --smearing mp --kpoints 8 8 8

# SCAN relaxation
fe-job -i slab.xyz -s qe --qe-task relax --qe-functional scan -o pw.in
```

Tasks: `scf`, `nscf`, `bands`, `relax`, `vc-relax`, `md`, `vc-md`. Functionals map to `input_dft` (PBE uses the pseudopotential's own functional). Spin reuses the same `guess_spin` estimator → `nspin` / `tot_magnetization`. Pseudopotentials are referenced as `<Element>.UPF` in `--pseudo-dir` (UPF files are user-supplied; the CP2K GTH database does not apply to QE).

Charge & spin (all targets)

Flag	Effect
`--charge INT`	Override total system charge (applied before spin estimation)
`--multiplicity INT`	Force 2S+1 — highest priority, disables auto-spin
`--auto-spin`	Estimate multiplicity from structure (details)

See the CLI Reference for the complete flag tables.

Chapter 16

Unpaired-Electron / Spin-Multiplicity Estimation

When generating a QC input file you must specify the system's **spin multiplicity** (2S+1). Guessing wrong produces a calculation that either fails to converge or converges to the wrong electronic state. `ferro` can estimate the multiplicity directly from the structure via `ferro_core::guess_spin`, exposed on every job builder through the `--auto-spin` flag.

The estimator uses three strategies in decreasing order of reliability and falls back automatically:

1. **Magnetic-moment sum** — most reliable
2. **Oxidation state + Hund's rule** — for ionic solids
3. **Electron-count parity** — universal lower bound

The result is always reconciled against the total electron count; an inconsistency triggers a warning and a fall back to the parity bound.

Strategy 1 — Magnetic-moment sum

If any atom carries a `magmom` (read from a QE input, extxyz, or VASP OUTCAR), the number of unpaired electrons is

$$n_{\text{unpaired}} = \text{round!}\left(\left|\sum_i m_i\right|\right), \qquad 2S+1 = n_{\text{unpaired}} + 1$$

This reflects an actual (DFT- or user-supplied) spin state and is preferred whenever magmom data is present.

Strategy 2 — Oxidation state + Hund's rule

For ionic solids with no magmom data, `assign_oxidation_states` assigns formal oxidation states by electronegativity rules and charge balance:

1. The most electronegative element that has a negative common oxidation state is the **anion**; it takes its most-negative common state (O → −2, F → −1, S → −2, …).
2. Remaining elements are **cations**. The combination of their common positive oxidation states that satisfies overall charge balance is selected (highest-oxidation solution preferred when ambiguous).

Each ion's unpaired count is then:

- **Transition metals** — d-electron count n_d = group − oxidation state; high-spin filling of 5 d-orbitals (Hund's rule):

$$n_{\text{unpaired}} = \begin{cases} n_d & n_d \leq 5 \\ 10 - n_d & n_d > 5 \end{cases}$$

- **Main group** — valence electrons after ionization filled into the s/p shell; closed-shell ions give 0.

Contributions are summed over all sites (ferromagnetic assumption → upper bound).

Strategy 3 — Electron-count parity

With no usable structural information, only a bound is given from the total electron count $N_e = \sum_i Z_i - q$:

- N_e odd → at least one unpaired electron (doublet, 2S+1 = 2)
- N_e even → singlet assumed (2S+1 = 1)

This is always applied as a final sanity check: if the multiplicity from strategy 1 or 2 has the wrong parity relative to N_e, a warning is emitted and the parity bound is used instead.

Worked examples

ZnP O — diamagnetic

Ion	Configuration	Unpaired
Zn^2	group 12, $n_d = 12 - 2 = 10 \rightarrow d^1$	0
P	main group, 5 − 5 = 0 valence e	0
O^2	6 + 2 = 8 → closed octet	0

Oxidation states resolve uniquely (Zn fixed at +2 P = +5). Total = **0 unpaired → multiplicity 1**.

MnS — high-spin d

S (electronegativity 2.58) is the anion at −2 Mn = +2. Mn^2 : group 7, $n_d = 7 - 2 = 5 \rightarrow$ d high-spin → **5 unpaired → multiplicity 6**.

Fe O

O $= -6$ 2 Fe $= +6$ Fe $= +3$. Fe^{3} : $n_d = 8 - 3 = 5 \rightarrow 5$ unpaired each $\rightarrow$ **10 total** $\rightarrow$ **multiplicity 11**.

Usage

```
# Auto-guess for a transition-metal oxide (CP2K)
fe-job -s cp2k -i Fe2O3.cif --auto-spin --smear

# Auto-guess for QE
fe-job -s qe -i Fe2O3.cif --auto-spin --kpoints 4 4 4

# Manual override (highest priority - disables auto-spin)
fe-job -s gaussian -i radical.xyz --charge 0 --multiplicity 2
```

Priority: explicit `--multiplicity` > `--auto-spin` (or builder default) > value from the input file. In the CP2K and QE builders `auto_spin` is on by default; passing `--multiplicity` disables it so the manual value is respected.

Limitations

These are inherent to formal-charge / parity methods and are reported as warnings:

- **Transition metals are always estimated high-spin.** Low-spin complexes (strong-field ligands) and the geometry-dependent crossover are not detected — verify with DFT.
- **Multi-centre magnetic coupling** (ferro- vs antiferromagnetic) cannot be inferred from structure; the sum is an upper bound.
- **Purely covalent molecules** fall back to the parity bound. O , for example, is reported as a singlet — its triplet ground state arises from * orbital degeneracy, which requires molecular-orbital theory.
- **Lanthanide f-electrons** are not counted.
- **Covalent transition-metal complexes** (organometallics) do not satisfy the ionic assumption.

The estimate is an *initial guess*, not a substitute for an electronic-structure calculation.

Related

- Job Builders — Gaussian / CP2K / QE input generation
- CLI Reference: `fe-job`

Part IV

Python API

Chapter 17

Python Bindings

`ferro` ships first-class Python bindings (PyO3) covering the core workflow: **read → transform → analyze**. The pure-Rust crates have zero Python awareness; the bindings live in the separate `ferro-python` crate, which is its own Cargo workspace and is built with maturin rather than `cargo build`.

Install

```
pip install maturin
cd ferro-python
maturin develop                  # build + install into the active venv/conda env
# or, without an active environment:
maturin build --release --interpreter "$(which python)"
pip install target/wheels/ferro-*.whl
```

> `cargo build` from the workspace root does **not** build this crate (and a standalone `cargo build` fails at the macOS link step on undefined Python symbols — that is expected; maturin supplies the correct link flags).

Quick start

```
import ferro

t = ferro.read("traj.lammpstrj", metal_units=True)   # -> Trajectory
print(len(t), t.n_atoms(), t.elements())

sc = ferro.supercell(t, 2, 2, 1)
ferro.write(sc, "POSCAR")

g = ferro.gr(t, r_max=10.0, dr=0.02)         # dict[str, list[float]]
d = ferro.msd(t, dt=2.0, elements=["Li"])
```

`Trajectory`

Returned by `ferro.read`; wraps the Rust `Trajectory` (single-frame files use this type too).

Method	Returns	Description
`n_frames()`	`int`	Number of frames
`n_atoms()`	`int | None`	Atom count (None if empty)
`elements()`	`list[str]`	First-frame elements, first-appearance order
`positions(frame)`	`list[(float,float,float)]`	Cartesian coords [Å]
`symbols(frame)`	`list[str]`	Element symbols (matches `positions`)
`cell(frame)`	`list[list[float]] | None`	3×3 cell rows [Å]; None if non-periodic
`charge(frame)`	`int`	Total charge
`multiplicity(frame)`	`int`	Spin multiplicity 2S+1
`tail(n)`	`Trajectory`	New trajectory with the last `n` frames
`len(t)`	`int`	= `n_frames()`

Frame indices are 0-based; out-of-range raises `IndexError`.

I/O

`read(path, metal_units=False) -> Trajectory`

Format auto-detected from the file name / extension:

Extension / name	Format
`.xyz` / `.extxyz`	XYZ / extended XYZ
`.pdb` / `.cif`	PDB / CIF
`POSCAR*` / `CONTCAR*`	VASP
`.in` / `.qe`	Quantum ESPRESSO input
`.inp` / `.restart`	CP2K input / restart
`.lammpstrj` / `.dump` / `.lammps`	LAMMPS dump (`metal_units` switches real metal)
`.data` / `.lmp`	LAMMPS data

`write(traj, path, metal_units=False)`

Writes by extension: `xyz`, `extxyz`, `pdb`, `cif`, `POSCAR`, `in`/`qe`, `data`/`lmp`, `lammpstrj`/`dump`.

Structure operations

Function	Description
`supercell(traj, nx, ny, nz)`	Per-frame supercell → new `Trajectory`
`add_vacuum_layer(traj, axis, thickness)`	Add vacuum along `"x"`/`"y"`/`"z"`
`merge(a, b, axis, gap)`	Merge first frames of two trajectories

Analysis

Analysis functions return `dict[str, list[float]]` (PyO3 converts the Rust `HashMap<String, Vec<f64>>` automatically — no NumPy dependency).

`gr(traj, r_max=None, dr=0.01, r_cut=2.3, r_min=0.005)`

Radial distribution function and coordination number. `r_max=None` auto-selects half the shortest cell vector of the first frame.

Returned keys:

- `"r"` — bin-centre radii [Å]
- `"gr:<El1-El2>"`, `"gr:total"` — partial / total g(r)
- `"cn:<center-neighbor>"` — directed cumulative CN(r)

```
g = ferro.gr(t, r_max=8.0, dr=0.05)
import matplotlib.pyplot as plt
plt.plot(g["r"], g["gr:total"])
```

`msd(traj, dt=1.0, shift=1, tau=None, elements=None)`

Mean squared displacement (time-origin averaged, NPT-safe).

Returned keys: `"time"` [fs], `"msd"` (total), `"msd_a"`, `"msd_b"`, `"msd_c"` (crystal axes for periodic systems, x/y/z otherwise).

```
d = ferro.msd(t, dt=2.0, elements=["Li"])
```

Errors

Rust errors (I/O failures, missing cells, empty trajectories) surface as Python exceptions (`RuntimeError` / `IndexError` / `ValueError`) with the underlying message preserved.

Part V

Reference

Chapter 18

CLI Reference

ferro provides six command-line binaries. All trajectory binaries share common flags for frame selection and parallelism. Run any binary without `-i` to print mode-specific help.

Common Flags

Flag	Description
`-i <file>`	Input file
`-o <file>`	Output file (default: mode-specific name)
`--last-n N`	Use only the last N frames
`--ncore N`	Parallel threads (default: all cores)
`--metal-units`	LAMMPS metal units (velocities Å/ps, forces eV/Å)

`fe-convert`

Format conversion.

```
fe-convert -i input.xyz -o output.pdb
fe-convert -i input.cif -o POSCAR
```

Supported input formats: `.xyz`, `.pdb`, `.cif`, LAMMPS dump
Supported output formats: `.xyz`, `.pdb`, `POSCAR`, LAMMPS dump

`fe-info`

Print structure summary (cell parameters, atom counts, element list).

```
fe-info -i input.xyz
fe-info -i traj.dump --last-n 1
```

fe-job

Generate QC software input files for **Gaussian**, **CP2K**, or **Quantum ESPRESSO**. Run without `-s` to see an overview; run with `-s <software>` but without `-i` to see software-specific help. See Job Builders for a guided overview.

```
fe-job                                          # overview
fe-job -s cp2k                                  # CP2K-specific help
fe-job -i input.xyz -s gaussian -m B3LYP -b 6-31G* -o job.gjf
fe-job -i input.xyz -s cp2k --task geo-opt --functional pbe --dispersion d3bj
fe-job -i Fe2O3.cif -s qe --auto-spin --kpoints 4 4 4 -o pw.in
```

Charge / Spin (shared by all targets)

Flag	Default	Description
`--charge`	(from file)	Override total system charge (applied before spin estimation)
`--multiplicity`	(from file)	Force spin multiplicity 2S+1; highest priority, disables auto-spin
`--auto-spin`	off (on by default for cp2k/qe)	Estimate multiplicity from structure

The estimator (magmom → oxidation state + Hund → electron parity) is documented in Spin Estimation.

Gaussian

Flag	Default	Description
`-m <method>`	(required)	DFT method, e.g. `B3LYP`, `PBE0`
`-b <basis>`	(required)	Basis set, e.g. `6-31G*`, `def2-TZVP`
`-o <file>`	`job.gjf`	Output file

CP2K

Task & Electronic Structure

Flag	Default	Candidates
`--task`	`energy`	`energy`, `force`, `geo-opt`, `cell-opt`, `md`, `freq`

Flag	Default	Candidates
`--functional`	`pbe`	`pbe`, `blyp`, `pbe0`, `b3lyp`, `revpbe`, `pbesol`, `scan`, `r2scan`, `hse06`
`--cp2k-basis`	`dzvp-molopt-sr`	`dzvp-molopt-sr`, `tzvp-molopt`, `tzv2p-molopt`, `dzvp-gth`, `tzvp-gth`, `pob-dzvp`, `pob-tzvp` (all-electron), or any custom string
`--dispersion`	`none`	`none`, `d3`, `d3bj`
`--scf`	`diag`	`diag` (metals/large), `ot` (insulators)
`--pbc`	(auto)	`xyz`, `z`, `none`; auto-detected from cell if omitted
`--kpoints`	(none)	Three integers, e.g. `--kpoints 2 2 2`
`--cutoff`	`400`	Plane-wave cutoff [Ry]
`--rel-cutoff`	`50`	Relative cutoff [Ry]
`--smear`	off	Enable Fermi–Dirac smearing

Output

Flag	Default	Candidates
`--atom-charge`	`none`	`none`, `mulliken`, `hirshfeld`, `hirshfeld-i`
`--cube`	`none`	`none`, `density`, `elf`, `hartree`
`--molden`	off	Export Molden orbital file
`--project`	`ferro`	CP2K project name

MD (only with `--task md`)

Flag	Default	Description
`--md-steps`	`10000`	Number of MD steps
`--md-timestep`	`1.0`	Timestep [fs]
`--temperature`	`298.15`	Temperature [K]
`--thermostat`	`csvr`	`csvr`, `nose`, `langevin`, `none`
`--traj-freq`	`100`	Trajectory write frequency [steps]
`--barostat`	off	Enable NPT barostat

Basis-set and pseudopotential names are resolved **per element** from a 2829-entry database (PBE / SCAN / all-electron, with matching valence `q`). `--cp2k-basis` selects the family; the exact element-specific name is filled in automatically. See Job Builders.

Quantum ESPRESSO

```
fe-job -i crystal.cif -s qe
fe-job -i metal.cif -s qe --smearing mp --kpoints 8 8 8
fe-job -i slab.xyz -s qe --qe-task relax --qe-functional scan -o pw.in
```

Flag	Default	Candidates / Description
`--qe-task`	`scf`	`scf`, `nscf`, `bands`, `relax`, `vc-relax`, `md`, `vc-md`
`--qe-functional`	`pbe`	`pbe`, `pbesol`, `revpbe`, `blyp`, `scan`, `r2scan`, `pbe0`, `hse06`
`--ecutwfc`	`50`	Plane-wave cutoff [Ry]
`--smearing`	`none`	`none`, `gaussian`, `mp`, `mv`, `fd` (mp/mv for metals)
`--kpoints`	(Gamma)	Three integers → Monkhorst-Pack mesh
`--pseudo-dir`	`./pseudo`	Pseudopotential directory (`<El>.UPF`)
`--md-steps`	`10000`	MD steps (`--qe-task md`/`vc-md`)
`--temperature`	`298.15`	MD target temperature [K]
`-o <file>`	`pw.in`	Output file

`ibrav = 0`; the cell is written as `CELL_PARAMETERS angstrom` from the structure. Spin uses the shared estimator → `nspin` / `tot_magnetization`.

---

fe-traj

Structural analysis of MD trajectories.

```
fe-traj -m <mode> -i traj.dump [flags] -o output
```

Modes

gr — Radial Distribution Function

Computes partial and total $g(r)$ plus coordination numbers $\text{CN}(r)$.

```
fe-traj -m gr -i traj.dump --r-max 10.0 --dr 0.01 --r-cut 2.3 -o gr.dat
```

Flag	Default	Description
`--r-max`	10.005	Maximum radius [Å]
`--dr`	0.01	Bin width [Å]
`--r-cut`	2.3	CN integration cutoff [Å]

Output: `<stem>.dat` (g(r)) and `<stem>_cn.dat` (coordination numbers)

`sq` — Structure Factor

Computes $S(q)$ via Fourier transform of $g(r)$.

```
fe-traj -m sq -i traj.dump --q-max 25.0 --dq 0.05 --weighting xrd -o sq.dat
```

Flag	Default	Description
`--q-max`	25.0	Maximum q [Å ^{1}]
`--dq`	0.05	q bin width [Å ^{1}]
`--weighting`	`both`	`xrd`, `neutron`, or `both`

`msd` — Mean Squared Displacement

Computes MSD and directional components; supports NPT and non-periodic trajectories.

```
fe-traj -m msd -i traj.dump --dt 2.0 --shift 10 --elements Li -o msd.dat
```

Flag	Default	Description
`--dt`	1.0	Timestep [fs]
`--shift`	1	Origin spacing [frames]
`--elements`	(all)	Comma-separated element filter

`angle` — Bond Angle Distribution

Computes $P(\theta)$ for all A–B–C triplets.

```
fe-traj -m angle -i traj.dump --r-cut-ab 2.3 --r-cut-bc 2.3 --d-angle 0.1 -o
↪  angle.dat
```

Flag	Default	Description
`--r-cut-ab`	2.3	A–B bond cutoff [Å]
`--r-cut-bc`	2.3	C–B bond cutoff [Å]
`--d-angle`	0.1	Histogram bin width [°]

`fe-corr`

Correlation function analysis.

```
fe-corr -m <mode> -i traj.dump [flags] -o output
```

Modes

`vacf` — Velocity Autocorrelation Function

Computes VACF and the running Green-Kubo diffusion integral.

```
fe-corr -m vacf -i traj.dump --dt 2.0 --elements Li --metal-units -o vacf.dat
```

Flag	Default	Description
`--dt`	1.0	Timestep [fs]
`--shift`	1	Origin spacing [frames]
`--tau`	(all)	Lag window [frames]
`--elements`	(all)	Element filter

Output columns: `time[fs]`, `vacf[v²]`, `vacf_x`, `vacf_y`, `vacf_z`, `diffusion[v²·fs]`

`rotcorr` — Rotational Autocorrelation

Computes $C_2(t)$ for molecular orientation vectors.

```
fe-corr -m rotcorr -i traj.dump --center P --neighbor O --r-cut 2.4 --dt 2.0 -o
↪  rotcorr.dat
```

Flag	Default	Description
`--center`	(required)	Central atom element
`--neighbor`	(required)	Neighbour atom element
`--r-cut`	1.2	Bond search cutoff [Å]
`--dt`	1.0	Timestep [fs]
`--shift`	1	Origin spacing [frames]
`--tau`	(all)	Lag window [frames]

Output columns: `time[fs]`, `C(t)`, `integral[fs]`

`vanhove` — Van Hove Self-Correlation

Computes $G_s(r, \tau)$ displacement histogram.

```
fe-corr -m vanhove -i traj.dump --tau 500 --dt 2.0 --r-max 8.0 --dr 0.02 -o
↪  vanhove.dat
```

Flag	Default	Description
`--tau`	(last frame)	Lag [frames]
`--dt`	1.0	Timestep [fs]
`--shift`	1	Origin spacing [frames]
`--r-max`	10.0	Max displacement [Å]
`--dr`	0.01	Bin width [Å]
`--elements`	(all)	Element filter

Output columns: `r[Å]`, `Gs(r,tau)`

---

fe-cube

3-D spatial distribution maps (Gaussian cube format).

```
fe-cube -m <mode> -i traj.dump [flags] -o output
```

Modes

density — Atomic Number Density

```
fe-cube -m density -i traj.dump --nx 80 --ny 80 --nz 80 --elements Li -o li.cube
```

Flag	Default	Description
--nx/ny/nz	50	Grid dimensions
--elements	(all)	Element filter

velocity — Mean Speed per Voxel

Requires trajectory with velocities (use --metal-units for LAMMPS metal dumps).

```
fe-cube -m velocity -i traj.dump --metal-units -o velocity.cube
```

force — Mean Force Magnitude per Voxel

Requires trajectory with forces.

```
fe-cube -m force -i traj.dump -o force.cube
```

radius — Hard-Sphere Occupancy

```
fe-cube -m radius -i traj.dump --elements Li --radius 0.7 --nx 100 --ny 100 --nz
↪  100 -o li_radius.cube
```

Flag	Default	Description
--radius	0.7	Hard-sphere radius [Å]
--nx/ny/nz	50	Grid dimensions
--elements	(all)	Element filter

sdf — Cluster SDF

```
fe-cube -m sdf -i traj.dump --qn 3 --former P --ligand O --cutoff-fl 2.4 \
        --modifier Zn --cutoff-ml 2.8 --grid-res 0.1 --sigma 1.5 -o sdf
```

Flag	Default	Description
--qn	3	Target Q_n level (0–3)
--former	P	Network-former element
--ligand	O	Bridging-ligand element
--cutoff-fl	2.4	Former–ligand cutoff [Å]

Flag	Default	Description
`--modifier`	(none)	Modifier cation element
`--cutoff-ml`	2.8	Modifier–ligand cutoff [Å]
`--grid-res`	0.1	Voxel size [Å]
`--sigma`	1.5	Gaussian broadening [voxels]
`--padding`	3.0	Grid margin [Å]
`--rmsd-warn`	0.5	RMSD warning threshold [Å]

Output: `<stem>_<label>.cube` per atom type (multiple families: `<stem>_fam<N>_<label>.cube`).

`chg-sdf` — Averaged Charge-Density SDF

Computes the orientationally averaged electron density around Qn clusters from a set of QE `pp.x` charge-density cube files. Does **not** require a trajectory `-i`; instead takes `--cubes`.

```
fe-cube -m chg_sdf \
    --cubes frame_000.cube frame_001.cube frame_002.cube \
    --qn 2 --former P --ligand O --cutoff-fl 2.4 \
    -o chg_sdf
```

Flag	Default	Description
`--cubes <files…>`	(required)	QE pp.x cube files, one per MD frame
`--qn`	3	Target Qn level (0–3)
`--former`	P	Network-former element
`--ligand`	O	Bridging-ligand element
`--cutoff-fl`	2.4	Former–ligand cutoff [Å]
`--modifier`	(none)	Modifier cation element
`--cutoff-ml`	2.8	Modifier–ligand cutoff [Å]
`--chg-padding`	6.0	Sub-grid boundary margin [Å]
`--rmsd-warn`	0.5	Alignment RMSD warning threshold [Å]

Output: `<stem>_Q<n>.cube` (one file per signature family).

See Averaged Charge-Density SDF for algorithm details.

`fe-bader`

Bader charge decomposition from DFT charge-density files. Supports VASP CHGCAR and Gaussian/QE cube files.

```
fe-bader -i CHGCAR                    # VASP CHGCAR
fe-bader -i charge.cube               # Gaussian/QE cube file
fe-bader -i CHGCAR -o bader           # custom output stem
fe-bader -i CHGCAR --method weight    # Yu-Trinkle weight method
```

Flag	Default	Description
`-i <file>`	(required)	Input file (`.cube` → cube reader; others → CHGCAR reader)
`-o <stem>`	`bader`	Output file stem
`--method`	`ongrid`	Bader method: `ongrid`, `neargrid`, `offgrid`, `weight`

Output Files

File	Content
`<stem>_ACF.dat`	Atomic Charges File — per-atom Bader charge, volume, min distance to surface
`<stem>_BCF.dat`	Bader Charge File — per-Bader-volume charge, volume, coordinates
`<stem>_AVF.dat`	Atomic Volume File — atom → Bader volume index mapping

---

fe-network

Glass network analysis: CN distribution, ligand classification (FO/NBO/BO/OBO), Qn speciation, modifier cation roles.

```
#
fe-network -i traj.dump --P-O=2.3

#     + xlsx
fe-network -i traj.dump --P-O=2.3 --Si-O=1.8 --format xlsx -o result.xlsx

#
fe-network -i traj.dump --P-O=2.3 --Zn-O=3.5 --modifier Zn

#
fe-network -i traj.dump --P-O=2.3 --Zn-O=3.5 --Na-O=3.2 --modifier Zn,Na
```

Pair Arguments

```
  --Former-Ligand=cutoff

--P-O=2.3     P-O   2.3 Å P    O
--Si-O=1.8    Si-O   1.8 Å
--Zn-O=3.5       --modifier Zn     -
```

`-i <file>`	—
`-o <file>`	`network`
`--format csv|xlsx`	`csv`
`--last-n N`	N
`--ncore N`	
`--metal-units`	LAMMPS metal
`--modifier Elem`	—

`<stem>_cn.csv`	CN	
`<stem>_ligand.csv`	FO / NBO / BO / OBO	
`<stem>_qn.csv`	Qn	
`<stem>_modifier.csv`	Free / T / B / M	`--modifier`

XLSX sheet CN Ligand Qn Modifier

Glass Network Analysis